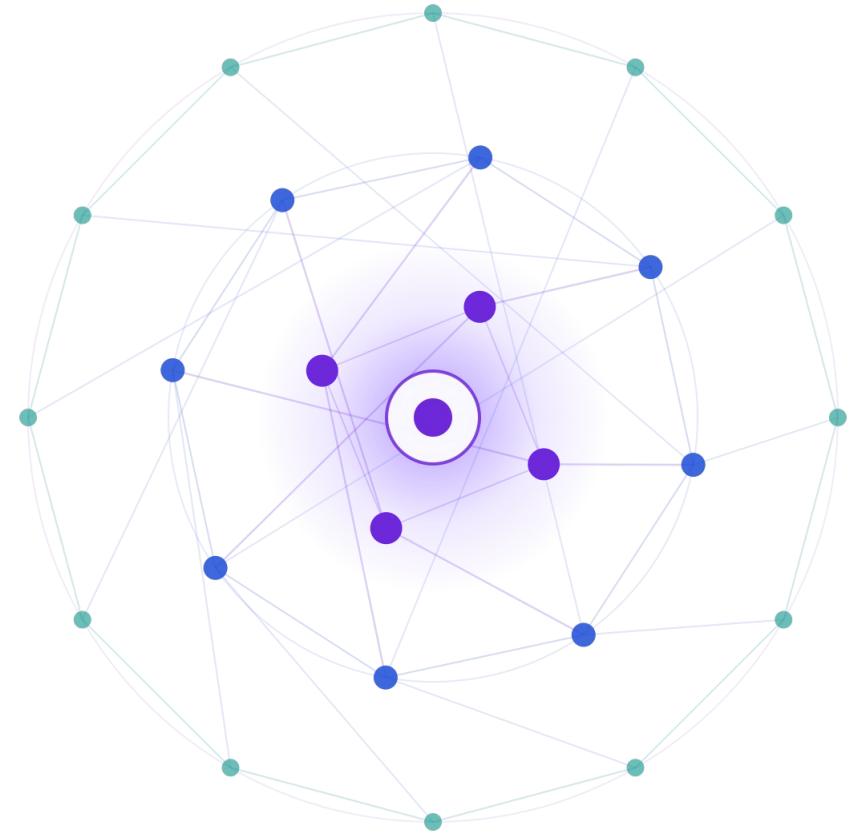


Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - 4 (Streaming Live RAG)
- Team Name - Pokermans
- College Name - VITV
- Member Name & Email 1- Priyank Garg, priyank.garg2024@vitstudent.ac.in
- Member Name & Email 2- Harshit Raj, harshit.raj2025@vitstudent.ac.in
- Member Name & Email 3- Abhyuday Rai, abhyuday.raai2024@vitstudent.ac.in
- Member Name & Email 4-
- Submission Github link - <https://github.com/abhyuday404/DuplexRAG>



Theme

Theme 04 · Streaming Live RAG

In a full-duplex voice conversation people speak one natural request, not a tidy query.

- **Hidden intents** - one utterance often carries several questions.
- **Latency** - waiting for the end of speech before searching leaves dead air.
- **Late details** - "actually it's fifty people" must refine the answer, not restart it.
- **Not every turn needs search** - "repeat that in two bullets" is formatting.
- **Trust** - every claim must be grounded in the provided corpus, and gaps must be said out loud.

Goal: a cheap, CPU-only engine that retrieves during speech, answers every hidden intent with citations, and keeps the answer alive across the conversation.

One utterance, three hidden questions

0.0 s "I need to plan a customer workshop in..."

0.8 s "...Pune for 30 people, and I need..."

1.6 s "...the cancellation policy and the catering options."

venue capacity
for 30

cancellation
terms

catering options

then: "Actually it's going to be fifty."

→ **refine the capacity claim, keep the rest (v1 → v2)**

then: "Can you repeat that in two bullets?"

→ **no retrieval, same citations**

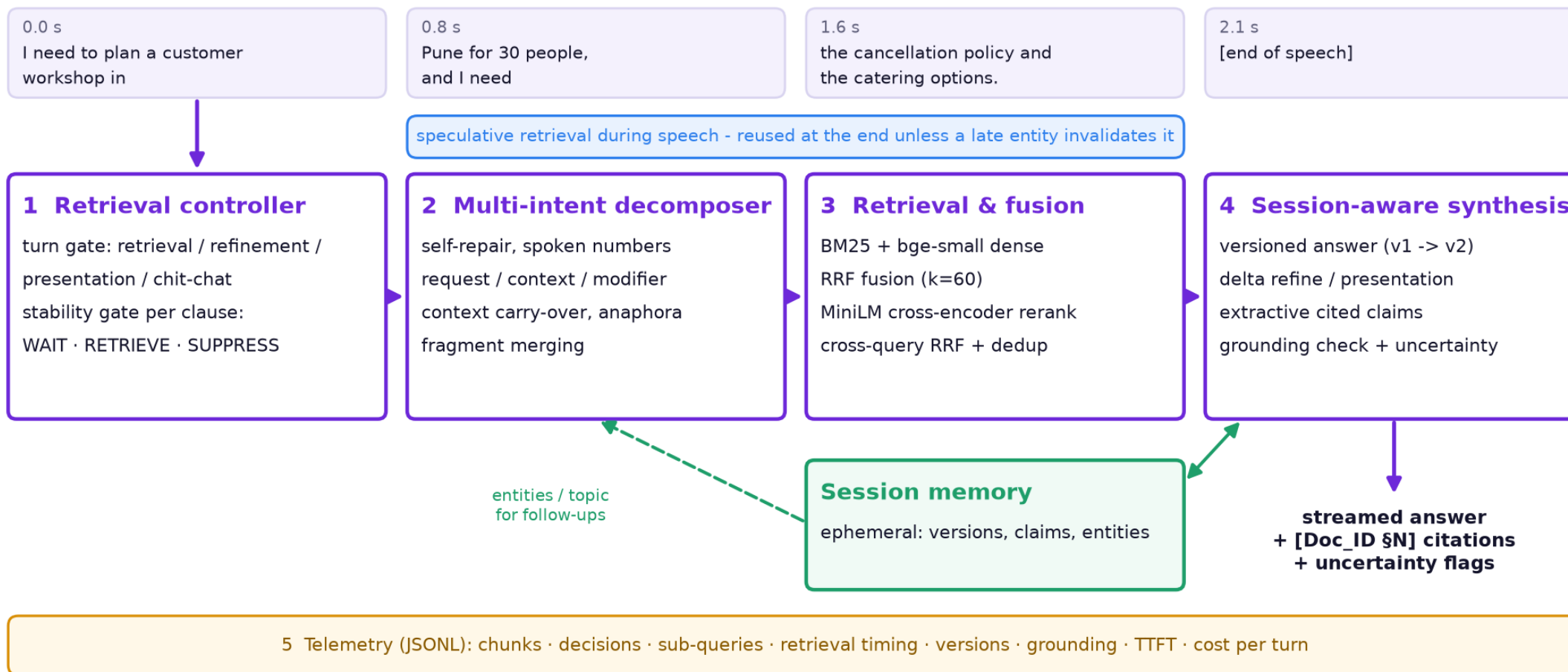
Existing Solutions & Gaps

Approach	How it works	Gap for full-duplex voice
Turn-based RAG	waits for end of speech, one query for the whole utterance	dead air while searching; compound requests get one blurred query
LLM query rewriting / multi-query (RAG-Fusion, HyDE)	an LLM rewrites or splits the query after the user stops	adds an LLM call to every turn: more latency and cost, and it can invent queries
Agentic RAG (plan → search → read loops)	multi-step tool use by an LLM agent	heavy orchestration and unpredictable latency - the brief asks for simple and cheap
Chat-history RAG	concatenates history into the next query	late details restart the search; context drifts; old citations are lost
Intent-based voice assistants	classify intent → fixed skill	no grounding in documents, no citations, no answer for long-tail policy questions

The gap: nothing retrieves during speech, refines the answer in place and proves every claim - on a CPU budget, without an LLM in the loop.

Our Solutions & Architecture Diagram

Incoming ASR stream (one natural request, several hidden questions)



Event-driven: on_chunk() per ASR partial, end_turn() at end of speech; retrieval jobs run in the background while the user keeps talking.

Demo & Product Walkthrough

The screenshot displays the DuplexRAG interface, which is divided into three main sections: CONVERSATION, GROUNDED ANSWER, and LIVE PIPELINE.

CONVERSATION: This section shows a live transcript of a conversation. The user's input is "Compound request + late detail + prese". The system's response is "So um I need to plan a customer workshop in Pune for about thirty people, and I also need the cancellation policy and, you know, whatever catering options there are." The transcript is segmented into chunks, with timestamps and status indicators (e.g., RETRIEVAL, LISTENING).

GROUNDED ANSWER: This section displays the system's response, which is grounded in the retrieved information. The answer is structured into sections: "Customer workshop in Pune for 30 people", "Cancellation policy", and "Catering options". Each section includes a summary and a list of retrieved documents (e.g., Doc_05 §2, Doc_03 §2, Doc_04 §2). A warning icon indicates that some information (catering options for Hinjewadi Tech Park Training Suites) could not be verified from the corpus.

LIVE PIPELINE: This section shows the internal state of the system, including the live pipeline and the event stream. The live pipeline includes components like "time to first token", "retrieval started before end of speech", "sub-queries (reused)", "claims grounded", "cost this turn (CPU)", and "v1 answer version". The event stream shows a sequence of events (e.g., retrieval_started, controller_decision, decomposition, retrieval_completed, evidence_fused, answer_started, first_token, answer_version, grounding_check, uncertainty_flag, turn_completed, turn_started) with their corresponding timestamps and details.

1. **Stream** replay a scenario, type, or speak (Chrome mic) - chunks arrive every ~1 s

2. **Watch the controller** WAIT / RETRIEVE / SUPPRESS per chunk; provisional and multi-intent searches fire mid-sentence

3. **Grounded answer** one line per hidden intent, every claim cited; click a citation to see the source section

4. **Late detail** "actually fifty people" → v2 with a diff: added / retained / retired claims, delta search only

5. **Format request** "two bullets" → no retrieval, same citations; the trace and cost are downloadable

Demo video:

[submission/DuplexRAG_demo.mp4](https://github.com/GoogleCloudPlatform/duplexrag/blob/main/submission/DuplexRAG_demo.mp4)

Tools and tech stack used

Models (CPU, ONNX)

- BAAI bge-small-en-v1.5 - 33M bi-encoder
- ms-marco-MiniLM-L-6-v2 - 22M cross-encoder
- fastembed + ONNX Runtime, 4 threads
- no GPU, no API keys, runs offline

Retrieval

- Okapi BM25 (NumPy, in-house)
- dense cosine search
- reciprocal-rank fusion (k=60)
- cross-encoder rerank + sentence scoring

Language layer

- rule-based spoken-language decomposer
- turn gate: rules + NumPy softmax classifier
- self-repair, spoken numbers, anaphora
- optional OpenAI-compatible LLM (verified)

Serving & UI

- FastAPI + WebSocket (uvicorn)
- vanilla JS live UI, Web Speech API mic
- CLI: index / serve / replay / ask / bench
- JSONL telemetry + JSON Schema

Engineering

- Python 3.12, uv lockfile
- Docker Compose (one command)
- pytest + smoke test (G1)
- git, periodic commits

Evaluation

- discrete-event streaming replay
- 4 independently written benchmark splits
- baseline + 7 ablations, gates G1-G6
- matplotlib figures, Playwright demo capture

Impact & Use case

Voice assistants

answer 'how do I...' questions from manuals, policies and settings docs while the user is still talking - with a source for every claim

Contact-centre agent assist

surface the right policy clauses during a live call; refine as the caller adds details; no dead air

Enterprise helpdesk (HR / IT / travel)

compound, messy questions answered in one turn with explicit 'not in the policy' flags

Hands-free field work

technicians and drivers ask by voice; CPU-only, offline-capable, cheap to run

3.2 s

median head start:
retrieval begins before the
user stops talking

7 ms

median time to first token
after speech

614 ms

same stack without
speculation

\$0.00007

compute cost per turn
(CPU only)

100%

claims supported by their
citation

0

fabricated citation IDs

held-out test split, laptop CPU (Ryzen 5 4600H)

Innovation highlights, results and limitations

Held-out test (30 sessions, 74 turns)	DuplexRAG	Baseline	Target
G2 retrieval starts before end of speech	100%	0%	$\geq 80\%$
G2 false triggers on no-retrieval turns	9%	100%	low
G3 compound turns: ≥ 2 intents isolated	83%	12%	$\geq 70\%$
G4 claims supported by citation	100%	100%	$\geq 85\%$
G4 fabricated citation IDs	0	0	0
G5 refinements: versioned, delta-only	88%	0%	pass
G6 telemetry trace coverage	100%	n/a	100%
Retrieval hit@3 (gold section)	89%	69%	
Key facts present in answer	56%	51%	
TTFT p50 after end of speech	7 ms	11 ms	

Baseline = turn-based RAG: waits for end of speech, one dense query, no rerank, no session memory. G1: one-command Docker build + automated replay; smoke test passes from a clean checkout.

Innovation highlights

- speculative per-clause retrieval with entity-aware reuse
- refine-in-place answer versions with claim diffs
- grounded by construction; claim-level verifier
- entity x aspect gaps said out loud
- no LLM needed: about \$0.0001 per turn

Limitations

- synthetic corpus (official corpus unavailable)
- extractive wording is less fluent than an LLM
- conservative 'not found': some unanswerables still get a related, cited answer
- speech simulated from transcripts (live mic in Chrome)

What's next

1

Plug in the official corpus + real ASR partials

streaming Whisper / on-device ASR; tune the stability gate on real partial-hypothesis revisions

2

Learned stability controller

train the retrieve / wait policy from ASR confidence, prosody and pauses

3

Small verified on-device LLM

fluent phrasing through the existing claim verifier; stream into TTS

4

Multilingual and code-mixed speech

Hindi-English, Korean; multilingual bi-encoder and cross-encoder

5

NPU / quantised models

int8 cross-encoder and cached sentence scores for sub-100 ms p95 on phones

6

Barge-in aware answers (with Theme 5)

stop, refine and resume the spoken answer when the user interrupts

Brownie points slide(differentiation)

Retrieves while you talk

median 3.2 s head start; searches fire mid-sentence and are reused

Zero fabricated citations

100% of claims verified against the cited section; IDs cannot be invented

CPU-only, about \$0.0001 per turn

two small ONNX models, no GPU, no API keys, works offline in Docker

Refine, don't restart

versioned answers with added / retained / retired claims; only the delta is searched

Says what it doesn't know

"catering for Hinjewadi Tech Park could not be verified" - entity x aspect coverage checks

Measured honestly

discrete-event replay; each held-out set written independently after a freeze; earlier held-out runs are reported too

Checklist- Updated on Public GitHub

- Working prototype code — public or shared GitHub repo (Y):
<https://github.com/abhyuday404/DuplexRAG>
- README with reproducible setup instructions (Y): README.md, Docker Compose, smoke test
- Demo video, max 5 minutes (Y): submission/DuplexRAG_demo.mp4 in the repo
- Presentation file (PPT or PDF) (Y): submission/VITV_Pokermans_Submission.pptx

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India